

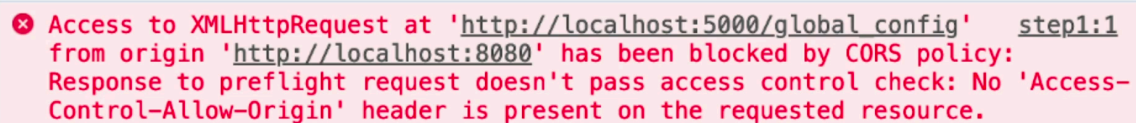
Section 5: Web Concepts in System Design

CORS: Cross-Origin Resource Sharing & Web Security

In this session, we will explore how browsers balance security and flexibility using CORS enabling modern web applications to communicate safely across the different domains without compromising user protection. So let's get started.

Why Does CORS Matters? (The Problem & Solution)

CORS matters because: Imagine your frontend is running on [app.com](#) and needs to call an API hosted on [api.com](#), technically the request is simple but the browser blocks it. The browser blocks it because the browser enforces the same origin policy, one of the web's most important security protections, without it a malicious website could potentially make requests on behalf of users to other sites that they are logged into.

A screenshot of a browser console error message. The message is displayed in red text on a light pink background. It starts with a red 'x' icon, followed by the text: 'Access to XMLHttpRequest at 'http://localhost:5000/global config' step1:1 from origin 'http://localhost:8080' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.'

```
✖ Access to XMLHttpRequest at 'http://localhost:5000/global config' step1:1  
  from origin 'http://localhost:8080' has been blocked by CORS policy:  
  Response to preflight request doesn't pass access control check: No 'Access-  
  Control-Allow-Origin' header is present on the requested resource.
```

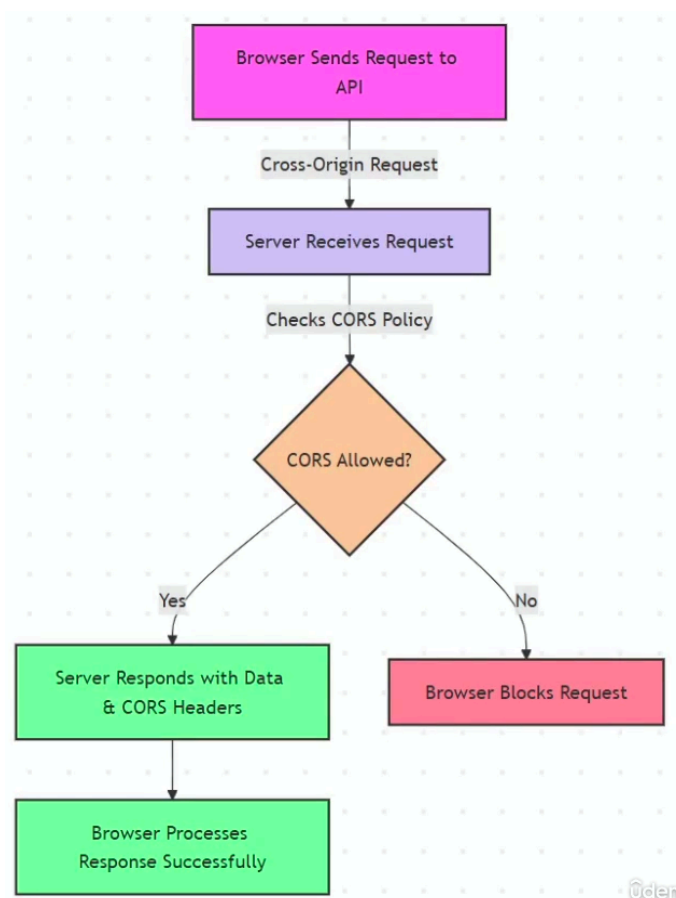
The challenge is that modern architecture rarely keeps everything under a single domain, frontend APIs, authentication services and third-party integration often distributed across different origins. So while CORS improves security, it can also prevent legitimate communication between trusted systems. CORS was introduced as the architectural solution to the same origin policy problem.

Instead of removing the browser security, it allows servers to explicitly declare which origins are allowed to access their resources. In other words, the browsers default behaviour is denied and CORS provides a controlled way to selectively grant access to some domains. Understanding this balance between security and flexibility is key to building modern applications that communicate safely across domains.

How CORS Works? (Request & Responses)

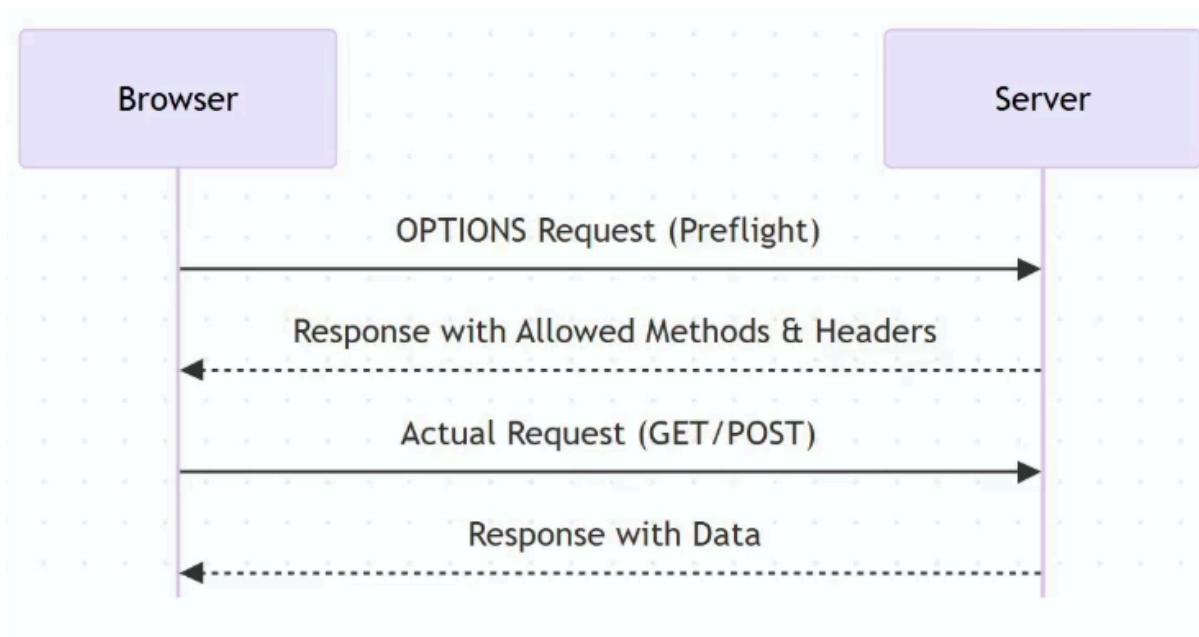
CORS often feels like a browser feature, but the key thing to remember is that it is actually controlled by the server. The browser simply enforces the rule. When a cross-origin request is made, the server must explicitly tell the browser whether the request is allowed or not. For straight-forward operations like reading data with a GET request, submitting a basic POST request. The browser sends the request directly. These are known as simple requests. But when an operation could potentially modify the resource, such as a PUT or DELETE request or when custom headers are involved. The browser becomes much more cautious.

Before sending the actual request, it performs a pre-flight check using the options request. Think of it as asking the server: If I send this request, will you allow it? The server responds with its CORS policy and only if the policy permits the operation, the browser proceeds with the real request.



The decision is driven by a set of response headers: Access-Control-Allow-Origin header specifies which domains are trusted, Access-Control-Allow-Methods which method operations are permitted and Access-Control-Allow-Headers determine which custom headers can be included. Together all these headers form a contract between the browser and the server. Enabling secure cross-origin communication without weakening the browser security model.

Preflight Requests & CORS Headers



Preflight requests are one of the most important parts of the CORS because they add an extra layer of security before potentially sensitive operations are performed. Rather than immediately sending a request that could modify data or include special headers, the browser first verifies whether the server is willing to accept it or not. This verification happens through OPTIONS request, you can think of it as a permission check. The browser is asking: if I send this request using these methods and these headers, will you allow it? And the server responds with its CORS policy.

This response contains key headers that define the rules:

- **Access-Control-Allow-Origin:** Identifies which origins are trusted. Example: `https://example.com`
- **Access-Control-Allow-Methods:** Specifies what operations are permitted such as: GET, POST.
- **Access-Control-Allow-Headers:** Lets any custom headers that clients are allowed to send such as: Authorization, Content-Type.

Only after the browser validates these headers does it send the actual request. This two step process helps to prevent unauthorised crossed origin interaction, while still allowing legitimate applications to communicate securely across different domains. As architects and backend-engineers correctly configuring these headers is very essential as a single misconfiguration can either break legitimate traffic or unintentionally exposing APIs to unwanted users.

Security Risks & Common Misconfiguration



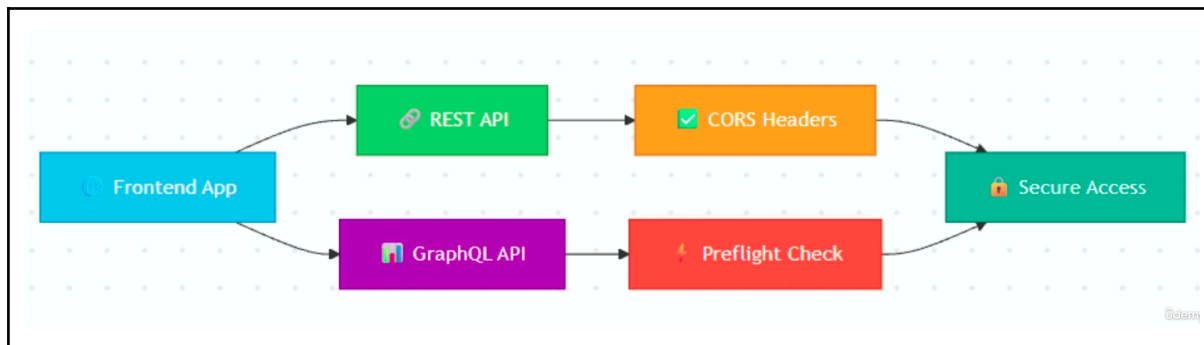
Like security mechanisms CORS is only as secure as its configuration. In the production system, the biggest risks usually don't come from CORS themselves, they come from overly permissive policies that were added for convenience and never tightened later. Common example is setting `Access-Control-Allow-Origin: *`, while this may seem an easy way to make things work during development, it effectively tells the browser that any website can access your API, for public resources that might be acceptable but for API serving user specific or sensitive data it is a serious security concern.

Another dangerous configuration is combining credentials with overly broad access roles. When authentication tokens, cookies or user sessions are involved, access should be restricted to explicitly trusted origins. Otherwise, you increase the risk of unauthorised access and data exposure. As your system grows, different APIs often have different security requirements. Public endpoints, partner APIs and internal services, should not share all the same CORS policy.

Experienced architects typically treat CORS as part of their overall API security strategy. Applying the principle of least privilege and granting only the access that is genuinely required. The safest approach is to maintain a whitelist of trusted origins, apply end-points specific policies where necessary and centralize enforcement to reverse proxies or API Gateways. This not only improves security but also makes CORS management far easier as your architecture evolves.

Handling CORS in APIs (REST & GraphQL)

Now from an application developer's perspective, CORS is not tight to an specific API style, it is a browser security requirement that applies whenever web clients communicate across origin. Whether you are building REST endpoint or GraphQL services, the browser expects the server to explicitly define what cross origin access is allowed.

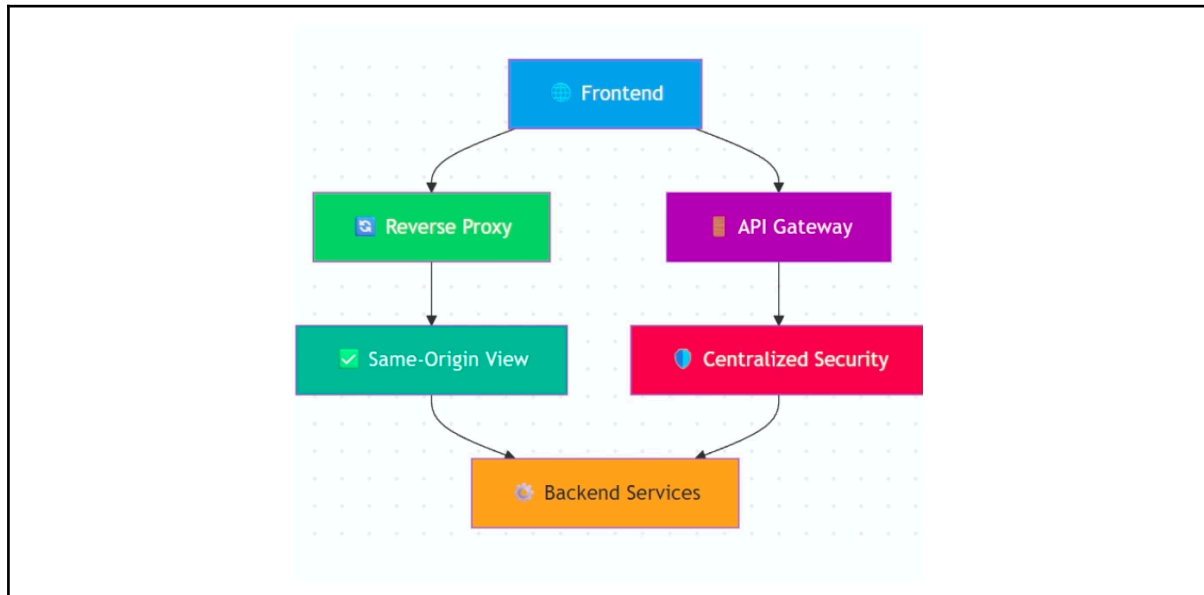


- **CORS in REST APIs:** CORS is extremely common in REST architecture as frontends and APIs are often deployed independently. A react application might run on one domain while the REST API runs on another domain. Frameworks such as express.js, spring boot and django provide built-in mechanisms for defining trusted origins, allowed methods and permitted headers, making CORS configuration a standard part of API deployment.
- **CORS in GraphQL APIs:** GraphQL faces the same challenge because it also operates over HTTP. Although GraphQL typically exposes a single endpoint, requests frequently include authentication headers, custom metadata or complex mutations which often triggers preflight checks. As a result GraphQL server must be configured with the same level of care as REST APIs.

The key architectural takeaway is that, CORS should be considered part of your API Gateway and security strategy rather than an after thought. Regardless of whether the backend exposes the REST resource of GraphQL schema, the goal remains the same which is allowing legitimate clients to communicate seamlessly while ensuring that browsers enforce strict access boundaries for everyone else.

Alternative to CORS & Role of API Gateways

Although CORS is the standard browser mechanism for controlling cross-origin access, experienced architects often solve the problem higher up in the architecture rather than relying solely on application layer CORS configuration.



- **Reverse Proxy:** A very common approach is to place a reverse proxy in front of your backend services. Instead of the browser calling multiple origins directly, all requests are routed through a single entry point. To the browser everything appears to come from the same origin so many CORS challenges disappear entirely. This is one reason why reverse proxies like Nginx are widely used in modern web architecture.
- **API Gateway Handling CORS:** As your system grows into dozens or even hundreds of services, managing CORS individually becomes very difficult, this is where API Gateway becomes valuable. Rather than configuring policies in every service manually, the gateway becomes the centralized CORS enforcement point; it can define trusted origins, allow-methods, authentication requirements and security rules consistently across the entire platform.

Beyond simplifying the management both approaches provide operational benefits.

- ➔ Reverse Proxies reduce complexity for frontend teams.
- ➔ API Gateways improves governance, observability and security.
- ➔ They can also optimize how preflight requests are handled, reducing unnecessary traffic and improving the overall responsiveness.

The architectural lesson is that CORS should not be viewed as just the browser configuration problems. In large scale systems proxies and API Gateways transform it into a centralized infrastructure concern and thereby making security policies easier to enforce, maintain and scale as the platform evolves.

Summary & Key Takeaways

- Same-Origin Policy (SOP) restricts cross-origin requests for security reasons.
- CORS is a server-side mechanism that allows controlled cross-origin access.
- Properly configured CORS prevents security risks.
- Alternatives like backend proxying can bypass CORS when needed.
- What's next:
 - Moving to Summary & Practical Applications of Web Concepts.

Interview Questions on CORS & Web Services

1. What is the Same-Origin Policy, and why does it exist?
2. How does CORS enable cross-origin requests?
3. What is a preflight request, and when is it required?
4. How do you configure CORS headers on a server?
5. What are common security risks associated with CORS?
6. What are alternatives to CORS for handling cross-origin requests?
7. How do API Gateways and Reverse Proxies help with CORS?